

UNIT IV : Analytical Questions

Slot C

Name : K. Lahari

Reg No. : 192325118

Course : Cryptography And Network Security

Course Code : CSA5104

Q1. IPSec architecture — AH and ESP secure communication model

Step 1 – Objective: Model how IPSec secures IP packets in transit, and show how Authentication Header (AH) and Encapsulating Security Payload (ESP) each contribute to that protection.

Step 2 – Identify the components: IPSec is built on three pieces: Security Associations (SAs) that hold the negotiated algorithms and keys, AH for authentication/integrity, and ESP for confidentiality plus (optionally) authentication/integrity.

Step 3 – Model AH: AH computes a keyed hash (ICV) over the immutable IP header fields and the payload, and adds a sequence number. The receiver recomputes the ICV; a mismatch means the packet was altered or spoofed, and an out-of-window sequence number means it is a replay.

Step 4 – Model ESP: ESP encrypts the payload (e.g., AES-GCM) and appends its own ICV over the ciphertext. This hides the payload from eavesdroppers and — using authenticated encryption — verifies integrity and origin in the same pass.

Step 5 – Map defences to attacks: Build a threat-to-mechanism table:

Attack	Defence	Protocol
Eavesdropping	Payload encryption	ESP
Tampering	Keyed hash (ICV)	AH, ESP
Spoofing	ICV covers header addresses	AH
Replay	Sequence number + window	AH, ESP

Step 6 – Conclusion: ESP with authenticated encryption in tunnel mode is the most efficient single mechanism, covering confidentiality, integrity, authentication,

and replay protection; AH is added only when header-level integrity beyond ESP's own coverage is required.

Q2. AH vs ESP — performance comparison

Step 1 – Define comparison criteria: Confidentiality, integrity, authentication, packet overhead, computational cost.

Step 2 – Set up the experiment: Run identical traffic (e.g., iperf throughput test, ping latency test) over three configurations: no IPSec, AH-only, ESP-only, on the same link.

Step 3 – Record results: Measure throughput (Mbps), latency (ms), and CPU utilisation for each configuration.

Property	AH	ESP
Confidentiality	Not provided	Provided
Integrity	Full packet incl. header	Payload + ESP header
Packet overhead	~12–24 bytes	Header+trailer+padding+ICV
Computational cost	Lower (hash only)	Higher (cipher+hash)
Latency impact	~2–5%	~8–15%

Step 4 – Interpret: AH's cost is dominated by hashing, so it scales gently with packet size; ESP's cost is dominated by encryption, so overhead grows faster with payload size and shrinks with hardware-accelerated AES-GCM.

Step 5 – Conclusion: Choose AH when header integrity without confidentiality is enough and overhead must stay minimal; choose ESP (with authenticated encryption) when confidentiality is required — which is the common case, making ESP the default choice in most deployments.

Q3. SA management system for multiple sessions

Step 1 – Objective: Support many concurrent secure sessions and evaluate resistance to replay and impersonation.

Step 2 – Define the data model: Security Association Database (SAD): keyed by (destination address, protocol, SPI); each entry holds keys, algorithms, sequence counter, anti-replay window, lifetime, mode. Security Policy Database (SPD): maps traffic selectors to bypass / discard / apply-IPSec-via-SA.

Step 3 – Handle session creation: On new traffic matching an SPD 'apply-IPSec' rule with no matching SAD entry, trigger IKE to negotiate a fresh SA pair (inbound/outbound) and insert it into the SAD.

Step 4 – Handle concurrency: Each SA's sequence counter and anti-replay window are independent, so many sessions run without interfering with each other's replay state.

Step 5 – Test against replay: Duplicate or delay a captured packet and resend it. Expected result: dropped, because its sequence number falls outside or repeats within the receiving SA's window.

Step 6 – Test against impersonation: Attempt to inject a packet under a spoofed source address without the negotiated SA key. Expected result: dropped, because the ICV computed with the attacker's (wrong) key fails verification.

Step 7 – Conclusion: Per-SA state plus authenticated key negotiation via IKE lets the system scale to many sessions while independently defeating replay and impersonation on each one; enforcing short SA lifetimes further bounds the damage of any single key compromise.

Q4. IPSec Transport mode vs Tunnel mode

Step 1 – Objective: Implement both modes for a given topology and compare packet formats and suitability.

Step 2 – Transport mode packet construction: Insert the AH/ESP header between the original IP header and payload; the original header is kept and used for routing.

Step 3 – Tunnel mode packet construction: Encapsulate the entire original IP packet as payload inside a new outer IP packet; the outer header carries the gateway addresses.

Feature	Transport mode	Tunnel mode
Original header	Preserved, visible	Hidden inside new packet
Overhead	Lower	Higher (extra IP header)
Typical use	Host-to-host	Gateway-to-gateway VPN

Step 4 – Apply to the topology: For two branch-office gateways linking private subnets over the Internet: use Tunnel mode so internal addressing stays hidden and policy applies per subnet. For two IPSec-aware servers needing direct end-to-end protection: use Transport mode to avoid the second IP header.

Step 5 – Conclusion: Mode choice follows from where the security boundary sits — at the end hosts (Transport) or at gateways protecting whole networks behind them (Tunnel).

Q5. Internet Key Exchange (IKE) — dynamic key management

Step 1 – Objective: Build a simplified IKE flow and compare it against static key distribution.

Step 2 – Phase 1 (IKE SA): Perform a Diffie–Hellman exchange combined with mutual authentication (pre-shared key or certificate) to establish a protected control channel, without ever transmitting the shared secret itself.

Step 3 – Phase 2 (IPSec SA): Inside the protected Phase-1 channel, negotiate SPI and algorithms for AH/ESP, then derive session keys from the Phase-1 master secret via a key-derivation function (optionally with an extra DH exchange for Perfect Forward Secrecy).

Step 4 – Rekeying: Before an SA's configured lifetime expires, IKE automatically negotiates fresh keys, so no session ever runs indefinitely on the same key.

Step 5 – Compare to static distribution: Static keys are manually configured, rarely rotated, and — if leaked — compromise all past and future traffic under that key. Manual redistribution to many endpoints is slow and error-prone.

Step 6 – Conclusion: Dynamic key management bounds key lifetime automatically, derives a fresh key per session, and (with PFS) isolates each session's compromise from all others — a materially stronger security posture than static keys, at the cost of the extra negotiation overhead.

Q6. Email header analysis — spoofing and phishing

Step 1 – Objective: Read a raw email header and identify the true sender, mail path, and forgery indicators.

Step 2 – Locate the sender fields: Compare From (displayed sender), Reply-To (where replies go), and Return-Path (bounce address) — legitimate mail usually keeps these consistent; spoofed mail often does not.

Step 3 – Trace the Received chain: Read Received headers bottom-to-top to reconstruct the path from originating server to inbox; the bottom-most entry is closest to the true origin.

Step 4 – Check authentication results: Inspect the Authentication-Results field for SPF (is the sending IP authorised for this domain?), DKIM (was the message cryptographically signed and unaltered?), and DMARC (do SPF/DKIM align with the From domain, and what policy applies on failure?).

Step 5 – Flag indicators: SPF/DKIM/DMARC failures, mismatched From/Reply-To domains, an implausible Received path, and disguised link destinations are the concrete indicators of spoofing or phishing.

Step 6 – Conclusion: Combining automated SPF/DKIM/DMARC verdicts with manual inspection of the Received chain and sender-field consistency reliably separates legitimate mail from spoofed or phishing attempts.

Q7. PGP-based secure email system

Step 1 – Objective: Design a PGP flow and evaluate the security services it provides.

Step 2 – Sender-side encryption: Generate a random session key; encrypt the message body with it using a symmetric cipher (AES); encrypt the session key itself with the recipient's public key.

Step 3 – Sender-side signing: Hash the message, sign the hash with the sender's private key, and attach the signature.

Step 4 – Recipient-side decryption: Decrypt the session key using the recipient's private key, then decrypt the message body with that session key.

Step 5 – Recipient-side verification: Recompute the hash of the decrypted message and verify it against the signature using the sender's public key.

Step 6 – Map to security services: Confidentiality: only the recipient's private key can recover the session key. Integrity: any alteration changes the hash and breaks verification. Authentication: a valid signature proves the sender held the claimed private key. Non-repudiation: the sender cannot credibly deny signing, so long as their private key was not compromised.

Step 7 – Conclusion: This combination of one-time symmetric encryption, asymmetric key wrapping, and digital signatures delivers all four core services without a central certificate authority — at the cost of relying on a decentralised web of trust for public-key validation.

Q8. PGP vs S/MIME

Step 1 – Objective: Implement/compare both for secure email and analyse the operational differences.

Step 2 – Compare trust models: PGP: web of trust, users sign each other's keys.
S/MIME: hierarchical PKI, certificates issued by a trusted CA.

Dimension	PGP	S/MIME
Key distribution	Manual / key servers	Via organisational PKI (e.g., LDAP/AD)
Scalability	Harder — trust propagates organically	Scales well inside an existing PKI
Enterprise fit	Common in technical communities	Native in Outlook and most enterprise clients

Step 3 – Evaluate scalability: PGP requires each user to build trust relationships manually, which does not scale cleanly to large organisations; S/MIME reuses an existing CA hierarchy that enterprises already maintain for other purposes.

Step 4 – Evaluate deployment cost: S/MIME integrates with certificates most enterprise mail clients already support; PGP requires separate tooling/plugins and user education on key management.

Step 5 – Conclusion: S/MIME is generally preferable inside a single organisation with an existing CA; PGP is preferable for cross-organisational or informal communication where no shared CA exists.

Q9. ML-based email spam detection

Step 1 – Objective: Build a classifier that separates spam from legitimate ('ham') email and evaluate it properly.

Step 2 – Collect and label data: Assemble a labelled dataset of spam and ham messages (e.g., a public corpus such as SpamAssassin or Enron-spam).

Step 3 – Extract features: Tokenise subject/body text and represent it as a bag-of-words or TF-IDF vector; optionally add header-based features (sender reputation, link count).

Step 4 – Train the classifier: Fit a Naïve Bayes model, estimating $P(\text{spam} | \text{tokens})$ from token frequencies under Bayes' theorem with a conditional-independence assumption; alternatively fit an SVM to find the maximum-margin separating hyperplane in the feature space.

Step 5 – Evaluate on a held-out set: Compute accuracy (overall correctness), precision (of flagged spam, how much is truly spam), recall (of all true spam, how much was caught), and F1-score (harmonic mean of precision/recall).

Step 6 – Interpret trade-offs: Because a false positive (legitimate mail marked spam) is usually costlier to the user than a false negative, precision is typically weighted more heavily than recall when tuning the classifier's decision threshold.

Step 7 – Conclusion: Naïve Bayes gives a fast, strong baseline on sparse text features; SVM often edges it out on accuracy at higher computational cost — the right choice depends on the precision/recall balance and latency budget required.

Q10. Integrated email security solution

Step 1 – Objective: Combine header analysis, spam filtering, phishing detection, and signature verification into one pipeline and evaluate its effectiveness.

Step 2 – Stage 1: header verification: Run SPF, DKIM, and DMARC checks at the mail gateway before delivery; reject or quarantine on hard failures.

Step 3 – Stage 2: spam filtering: Score the message with the ML classifier from Q9; route high-confidence spam to quarantine.

Step 4 – Stage 3: phishing detection: Check URLs/domains against reputation lists, rewrite links for click-time scanning, and flag look-alike domains impersonating known brands.

Step 5 – Stage 4: signature verification: For signed mail, verify PGP/S/MIME signatures to confirm sender authenticity and message integrity.

Step 6 – Combine into a risk score: Aggregate the four stage outputs into a single risk rating (e.g., weighted sum or rule-based escalation) rather than relying on any one stage's binary verdict.

Step 7 – Evaluate effectiveness: Because modern attacks blend techniques (e.g., a spoofed domain carrying a phishing link that also evades a spam filter), layered detection catches threats that any single stage would miss, improving overall detection coverage and reducing false negatives compared with a single-technique filter.

Step 8 – Conclusion: An integrated pipeline is more resilient than any individual defence because it forces an attacker to simultaneously defeat authentication checks, statistical spam detection, reputation-based phishing detection, and cryptographic signature verification — a materially higher bar than bypassing one filter alone.